



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE AND ALGORITHM SECJ2013

ASSIGNMENT 1: ANALYSIS OF SORTING ALGORITHMS

DUE DATE:
19/11/2024

GROUP:
8

NAME	MATRIC NO
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220
LAM YOKE YU	A23CS0233
MUHAMMAD HAFIZI SYAMIR BIN HERMAN	A22EC0213
TAN YI YA	A23CS0187

BUBBLE SORT ANALYSIS

1. DATA SAMPLE

Student marks = [75, 95, 60, 88, 70]

2. CODE C++

```
#include <iostream>
using namespace std;

void bubbleSort(int marks[ ], int a) {
    for (int m = 0; m < a - 1; m++) {
        // last element already sorted
        bool swapped = false; //flag to detect if any swap
        occurred in this pass
        for (int n = 0; n < a - m - 1; n++) {
            if (marks[n] > marks[n + 1]) {
                // Swap marks
                int tempMark = marks[n];
                marks[n] = marks[n + 1];
                marks[n + 1] = tempMark;
                swapped = true;
            }
        }
        // if no swaps occurred in this pass , array is sorted
        if ( ! swapped ) break;
    }
}

void printMarks( int marks[ ], int a) {
    for (int m = 0; m < a; m++) {
        cout << marks[m] << endl;
    }
}

int main() {
    int marks[ ] = {75, 95, 60, 88, 70};
    int a = sizeof(marks) / sizeof(marks[0]);

    cout << "ORIGINAL DATA\n";
    printMarks( marks, a);

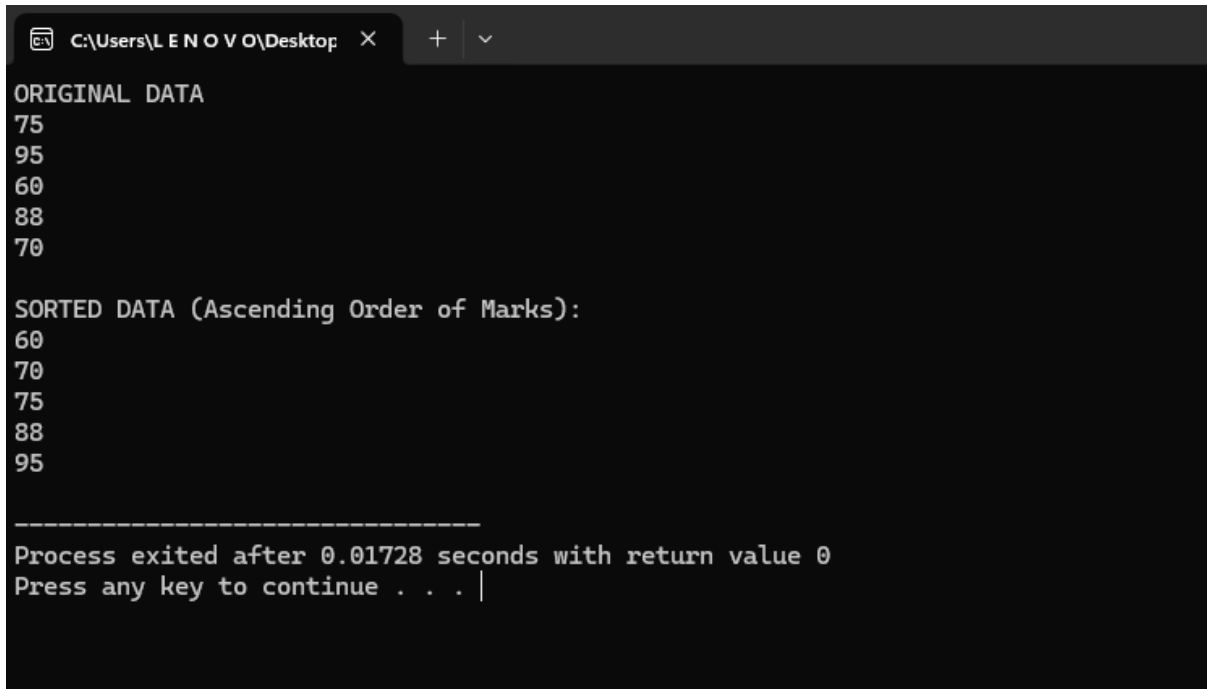
    bubbleSort(marks, a);

    cout << "\nSORTED DATA (Ascending Order of Marks):\n";
```

```
    printMarks( marks, a);  
  
    return 0;  
}
```

3. OUTPUT

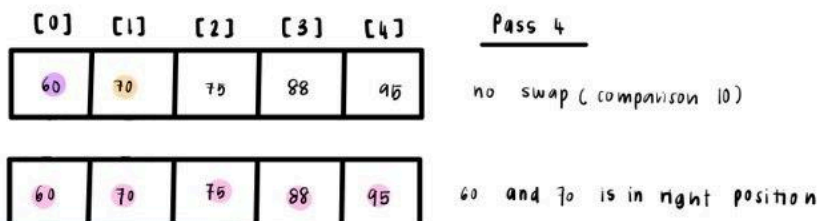
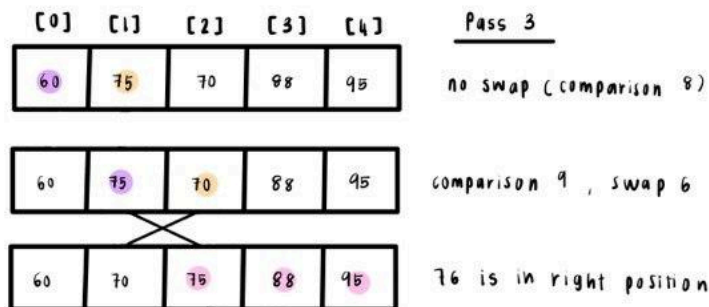
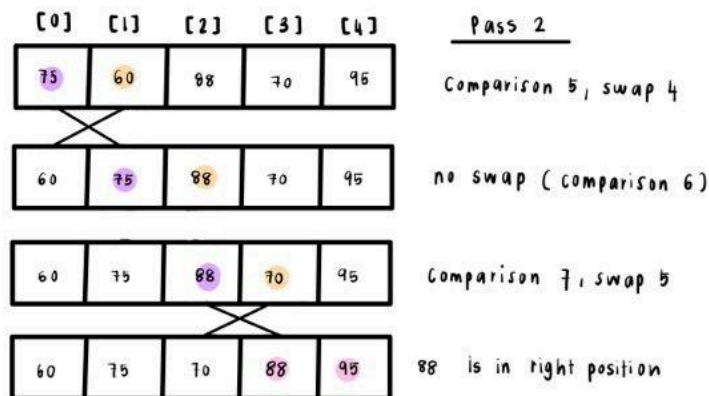
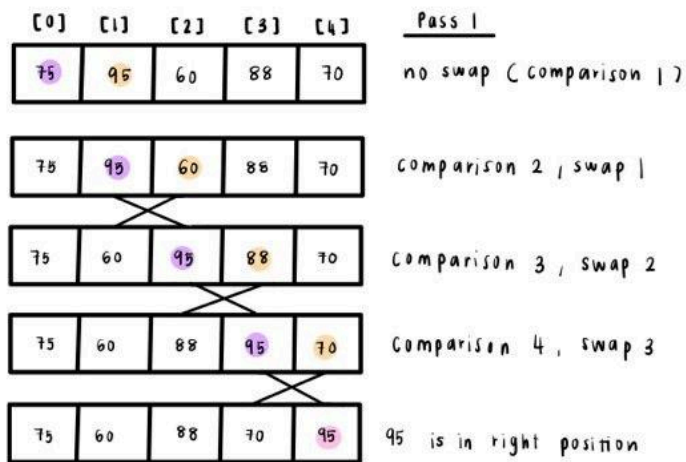
a. The sorted list of student marks.



```
C:\Users\LENOVO\Desktop  X  +  v  
ORIGINAL DATA  
75  
95  
60  
88  
70  
  
SORTED DATA (Ascending Order of Marks):  
60  
70  
75  
88  
95  
  
-----  
Process exited after 0.01728 seconds with return value 0  
Press any key to continue . . . |
```

b. The total number of comparisons and swaps for bubble sorting algorithm

- Made 10 comparisons
- Made 6 swaps



IMPROVED BUBBLE SORT ANALYSIS

1. DATA SAMPLE

Student marks = [75, 95, 60, 88, 70]

2. CODE C++

```
// Improved Bubble Sort
#include <iostream>
using namespace std;

void bubbleSort(int data[], int n){
    int temp;
    bool sorted = false;

    // Variables to count number of comparison and swaps
    int com = 0, swap = 0;

    for(int pass = 1; (pass < n) && !sorted; ++pass){
        sorted = true;
        for(int x = 0; x < n - pass; ++x){
            com++;
            if(data[x] > data[x+1]){
                // Exchange Items
                swap++;
                temp = data[x];
                data[x] = data[x+1];
                data[x+1] = temp;
                sorted = false;

                // Print array after swap
                cout << "Element swapped at pass = " << pass
                << " and x = " << x << "\n";
                for(int i = 0; i < n; i++){
                    cout << data[i] << " ";
                }
                cout << "\n\n";
            }
        }
    }

    // Print the comparisons and swaps
    cout << "Improved Bubble sort completed...\n"
    << "Comparisons made: " << com << "\n"
    << "Swaps made: " << swap << "\n";
}
```

```

int main() {
    int data[] = {95, 88, 75, 70, 60};
    int n = 5;

    bubbleSort(data, n);

    cout << "Sorted Array: ";
    for(int i = 0; i < n; i++){
        cout << data[i] << " ";
    }

    return 0;
}

```

3. OUTPUT

Element swapped at pass = 1 and x = 1
75 60 95 88 70

Element swapped at pass = 1 and x = 2
75 60 88 95 70

Element swapped at pass = 1 and x = 3
75 60 88 70 95

Element swapped at pass = 2 and x = 0
60 75 88 70 95

Element swapped at pass = 2 and x = 2
60 75 70 88 95

Element swapped at pass = 3 and x = 1
60 70 75 88 95

Improved Bubble sort completed...

Comparisons made: 10

Swaps made: 6

Sorted Array: 60 70 75 88 95

Improved Bubble Sort

	[0]	[1]	[2]	[3]	[4]	Pass	Sorted	x
	75	95	60	88	70			
	75	95	60	88	70	1	1	0
①	75	60	95	88	70		0	1
②	75	60	88	95	70		0	2
③	75	60	88	70	95		0	3
④	60	75	88	70	95	2	1	0
	60	75	88	70	95		0	1
⑤	60	75	70	88	95		0	2
	60	75	70	88	95	3	1	0
⑥	60	70	75	88	95		0	1
	60	70	75	88	95	4	1	0

Number of comparison: 10

Number of swaps: 6

SELECTION SORT

1. DATA SAMPLE

Student marks = [75, 95, 60, 88, 70]

2. CODE C++

```
#include <iostream>
using namespace std;

void selectionSort(int Data[], int n) {
    const int count = n;
    int r = n;
    r--;
    int comp = 0, Nswap = 0; //number of comparison and swap
    for (int last = n-1; last >= 1; --last) {
        int pass = count - (r--);
        cout << "Pass = " << pass << endl;
        //select largest item at array
        int largestIndex = 0;
        //largest item is assumed start at index 0
        for (int p=1; p<=last; ++p) {
            if (Data[p] > Data[largestIndex]) {
                largestIndex = p;
            } //end if
            comp++;
        } //end for
        cout << "Largest Index: " << largestIndex << endl;
        if (Data[largestIndex] == Data[last]) {
            cout << "No Swap" << endl;
        }
        else {
            //swap largest item Data[largestIndex] with
            Data[last]
            cout << Data[largestIndex] << " swapped with " <<
            Data[last] << endl;
            swap(Data[largestIndex], Data[last]);
            Nswap++;
        }
        for(int i = 0; i < n; i++) {
            cout << Data[i] << " ";
        }
        cout << endl << endl;
    } //end for

    cout << "Array Sorted Successfully" << endl;
```

```

        cout << "Number of Comparisons: " << comp << ", ";
        cout << "Number of Swaps: " << Nswap << endl;

    } //end selection sort

int main() {
    int data[] = {75, 95, 60, 80, 70};
    int n = 5;

    cout << "Original Array: ";
    for(int i = 0; i < n; i++) {
        cout << data[i] << " ";
    }
    cout << endl;

    selectionSort(data, n);

    cout << "Sorted Array: ";
    for(int i = 0; i < n; i++) {
        cout << data[i] << " ";
    }
    return 0;
}

```

3. OUTPUT

Original Array: 75 95 60 80 70

Pass = 1

Largest Index: 1

95 swapped with 70

75 70 60 80 95

Pass = 2

Largest Index: 3

No Swap

75 70 60 80 95

Pass = 3

Largest Index: 0

75 swapped with 60

60 70 75 80 95

Pass = 4

Largest Index: 1

No Swap

60 70 75 80 95

Array Sorted Successfully

Number of Comparisons: 10, Number of Swaps: 2

Sorted Array: 60 70 75 80 95

Process finished with exit code 0

Pass 1

[4]	70	70	70	70	95
[3]	88	88	88	88	88
[2]	60	60	60	60	60
[1]	95	95	95	95	70
[0]	75	75	75	75	75

comparison = 4

swap = 1

Pass 2

[4]	95	95	95	95
[3]	88	88	88	88
[2]	60	60	60	60
[1]	70	70	70	70
[0]	75	75	75	75

comparison = 7

swap = 1

Pass 3

[4]	95	95	95
[3]	88	88	88
[2]	60	60	75
[1]	70	70	70
[0]	75	75	60

comparison = 9

swap = 2

Pass 4

[4]	95	95
[3]	88	88
[2]	75	75
[1]	70	70
[0]	60	60

comparison = 10

swap = 2

INSERTION SORT

1. DATA SAMPLE

Student marks = [75, 95, 60, 88, 70]

2. CODE C++

```
#include <iostream>
using namespace std;

// Function to perform insertion sort
void insertionSort(int marks[], int a) {
    for (int i = 1; i < a; i++) {
        int key = marks[i]; // Store the current element
        int j = i - 1;

        // Move elements of marks[0..i-1] that are greater
        // than key to one position ahead
        while (j >= 0 && marks[j] > key) {
            marks[j + 1] = marks[j];
            j--;
        }

        // Place the key element at the correct position
        marks[j + 1] = key;
    }
}

// Function to print the marks
void printMarks(int marks[], int a) {
    for (int m = 0; m < a; m++) {
        cout << marks[m] << endl;
    }
}

int main() {
    int marks[] = {75, 95, 60, 88, 70};
    int a = sizeof(marks) / sizeof(marks[0]);

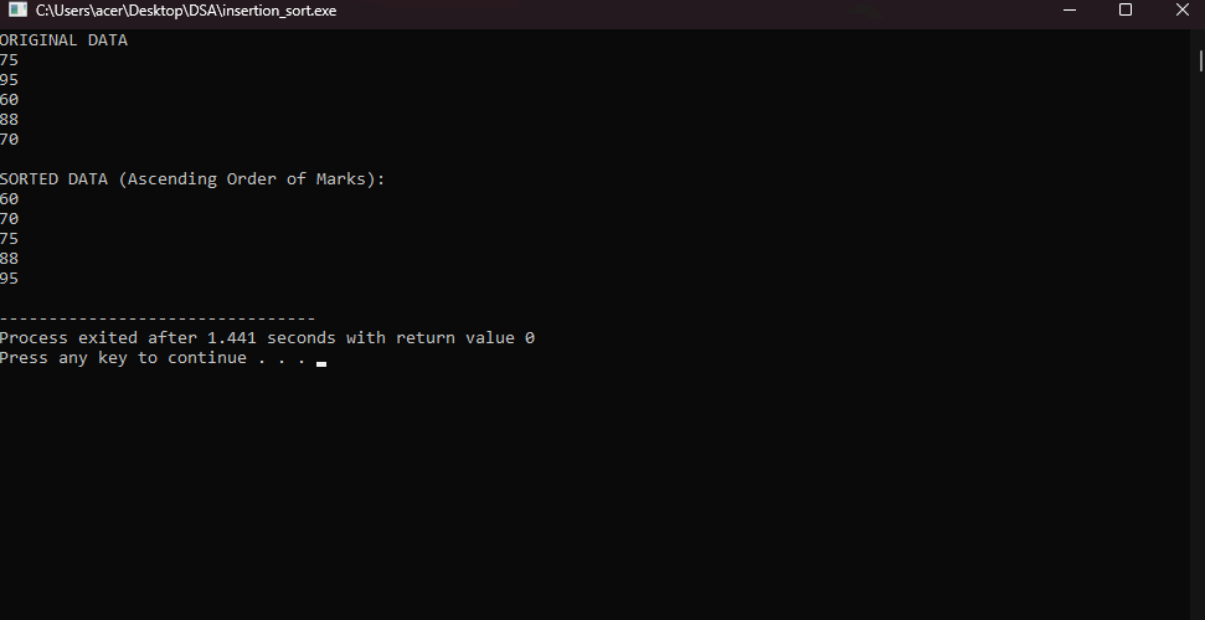
    cout << "ORIGINAL DATA\n";
    printMarks(marks, a);

    insertionSort(marks, a);

    cout << "\nSORTED DATA (Ascending Order of Marks):\n";
    printMarks(marks, a);
}
```

```
    return 0;  
}
```

3. OUTPUT



```
C:\Users\acer\Desktop\DSA\insertion_sort.exe  
ORIGINAL DATA  
75  
95  
60  
88  
70  
  
SORTED DATA (Ascending Order of Marks):  
60  
70  
75  
88  
95  
  
-----  
Process exited after 1.441 seconds with return value 0  
Press any key to continue . . .
```

Insert 95 in sorted portion: No comparisons or swaps

Insert 60:

- Compare 60 & 95: Shift. (1 comparison, 1 swap)
- Compare 60 & 75: Shift. (1 comparison, 1 swap)
- Place 60. Total: 2 comparisons, 2 swaps.

Insert 88:

- Compare 88 & 95: Shift. (1 comparison, 1 swap)
- Place 88. Total: 1 comparison, 1 swap.

Insert 70:

- Compare 70 & 95: Shift. (1 comparison, 1 swap)
- Compare 70 & 88: Shift. (1 comparison, 1 swap)
- Compare 70 & 75: Shift. (1 comparison, 1 swap)
- Place 70. Total: 3 comparisons, 3 swaps.

COMPARISON FOR ALL SORT

	Average Case		Best Case		Worst Case	
	data = [75, 95, 60, 88, 70]		d2 = [60, 70, 75, 88, 95]		d3 = [95, 88, 75, 70, 60]	
	Comparison	Swap	Comparison	Swap	Comparison	Swap
Bubble Sort	10	6	10	0	10	10
Improved Bubble Sort	10	6	4	0	10	10
Selection Sort	10	2	10	0	10	2
Insertion Sort	9	9	4	0	10	10

For each sort, 3 data sets of different arrangements and cases are used to test and count the number of comparisons and swaps. However, in the case of insertion sort, the elements of the array are shifted rather than swapped. In the average case, we can see that the number of comparisons in bubble sort, improved bubble sort and selection sort are similar, with insertion sort having a slightly fewer comparison.

In both bubble sort and selection sort, the number of comparisons between elements is the same across all three cases. This shows that the efficiency of both sorting methods does not depend on the initial arrangement of the data. These algorithms iterate through all elements repeatedly to make comparisons, resulting in a minimum of $(n - 1) + (n - 2) + \dots + 2 + 1 = O(n^2)$ comparisons.

In the best case, both the bubble sort and selection sort cannot may not be as efficient as the improved bubble sort and insertion sort. The improved bubble sort uses the `sorted` variable to keep track if the array is already sorted, reducing redundant comparisons. In insertion sort, the array on the left is always sorted, and the current item is compared to the previous elements until the correct position is found. Thus, in the best case, the number of comparisons will be only $n - 1$ times for both sorting methods. The sorting time and the number of comparisons between data in average case and best case can be minimised as they minimise redundant checks.

When comparing the number of swaps, it may not be so meaningful as the array is modified differently for the methods. Insertion sort shifts elements instead of swapping them, whereas bubble sort, improved bubble sort, and selection sort modify the array by swapping elements.

In conclusion, insertion sort would be more efficient in terms of number of comparisons. while selection sort would be more efficient in terms of number of swaps.

REFLECTION

Lam Yoke Yu

This assignment is quite interesting where we compare all the simple sorting methods introduced in the lecture. This assignment helped me to solidify my understanding of these sorting algorithms and its efficiency.

Tan Yi Ya

This assignment has improved my understanding of simple sorting methods once again by letting us do hands-on code. We are able to trace back each step in the sorting methods and learn with our teammates.

Muhammad Hafizi Syamir

This assignment has deepened my understanding about sorting methods. By implementing algorithms manually provides insight into workings and enhancing my programming skills.

Damiya Aina

By doing this assignment , I discover a lot of sorting techniques and having fun understanding each of them . But, I still need to keep doing more exercise related to the coding while i already understand these sort techniques by visualising the steps